# Static Program Analysis Guided LLM Based Unit Test Generation

Sujoy Roy Chowdhury*
Ericsson
Bangalore, India
sujoy.roychowdhury@ericsson.com

Giriprasad Sridhara*
Ericsson
Bangalore, India
giriprasad.sridhara@ericsson.com

A K Raghavan†
Independent Researcher
Chennai, India
oneraghavan@gmail.com

Joy Bose
Ericsson
Bangalore, India
joy.bose@ericsson.com

Sourav Mazumdar
Ericsson
Bangalore, India
sourav.mazumdar@ericsson.com

Hamender Singh
Ericsson
Bangalore, India
hamender.singh@ericsson.com

Srinivasan Bajji Sugumaran
Ericsson
Bangalore, India
srinivasan.bajji.sugumaran@ericsson.com

Ricardo Britto
Ericsson
Stockholm, Sweden
ricardo.britto@ericsson.com

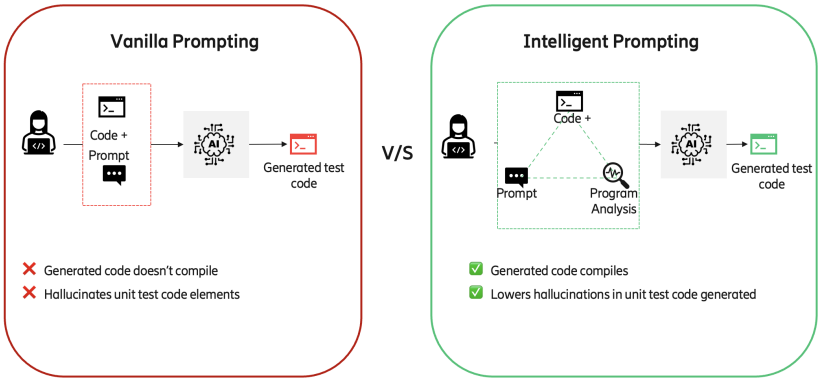


**Figure 1: Unit Test Case Generation enhanced with Program Analysis**

## Abstract

We describe a novel approach to automating unit test generation for Java methods using large language models (LLMs). Existing LLM-based approaches rely on sample usage(s) of the method to test (focal method) and/or provide the entire class of the focal method as input prompt and context. The former approach is often not viable due to the lack of sample usages, especially for newly written focal methods. The latter approach does not scale well enough; the bigger the complexity of the focal method and larger associated class, the harder it is to produce adequate test code (due to factors such as exceeding the prompt and context lengths of the underlying LLM). We show that augmenting prompts with *concise* and *precise* context information obtained by program analysis increases the effectiveness of generating unit test code through LLMs. We validate our approach on a large commercial Java project and a popular open-source Java project.

## CCS Concepts

• **Computing methodologies** → **Natural language generation**; *Discourse, dialogue and pragmatics*; • **Software and its engineering**;

## Keywords

Unit Test Case Generation, Large Language Models, Static Analysis

**ACM Reference Format:**
Sujoy Roy Chowdhury, Giriprasad Sridhara, A K Raghavan, Joy Bose, Sourav Mazumdar, Hamender Singh, Srinivasan Bajji Sugumaran, and Ricardo Britto. 2024. Static Program Analysis Guided LLM Based Unit Test Generation. In *8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD) (CODS-COMAD Dec '24), December 18–21, 2024, Jodhpur, India.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3703323.3703742

*Equal Contribution
†The author was at Ericsson R&D during this study



*CODS-COMAD Dec '24, Jodhpur, India*

ACM ISBN 979-8-4007-1124-4/24/12
https://doi.org/10.1145/3703323.3703742

## 1 Introduction

Unit tests are an essential part of software quality assurance [2]. They are complete functions (methods) with one or more statements that invoke the *method under test* (*also, Focal Method*). They have at least one assertion or verification statement about the result

accruing from calling the *Focal Method*. The advent of LLMs[5, 7, 11] has enabled researchers to explore their use for unit test generation [8, 10]. However, these approaches have limitations that prevent their usage for unit test generation in large commercial software. This paper discusses these shortcomings and proposes a novel approach to address them.

The approach described in [8] relies on *sample usages* of the *focal method* (i.e., method under test). In our use case, especially for newly written focal methods, such example usages are not readily available. Thus, we cannot use this approach.

An orthogonal approach to unit test generation using LLMs treats the problem as a code completion task [10]. It provides the entire source file in which the focal method is defined and the initial parts of an unit test file with comments describing that the test file contains unit tests. We use this approach as a *baseline* in our evaluation, and show that this approach leads to a large prompt and context and *unfortunately* it does not generate unit tests for many focal methods in our code base.

We alleviate the above problem by adroitly ensuring that the prompt and the context is as *precise* and *concise* as possible using static program analysis which we describe in Section 4.We augment the prompt with information obtained from static program analysis of the focal method. Our technique ensures that only relevant information is provided to the LLM and hence the context length is not exceeded enabling the LLMs to generate syntactically correct unit tests.

The main contributions of this paper are as follows:

(1) A novel and efficient approach for unit test generation using LLMs whose prompts are enhanced *precisely* and *concisely* with static program analysis; and,
(2) An evaluation of the above approach on a commercial closed source Java Project and an open source Java project and comparing against the baseline approach [10].

The remainder of this paper is organized as follows. Section 2 describes the related work while Section 3 provides a high level background about the static program analysis used in this paper. Section 4 describes our methodology and Section 5 depicts our evaluation. Section 6 delineates our results. In Section 7 we provide a discussion of the results. We conclude in Section 8 along with a discussion of future work.

## 2 Related Work

RLPG (Repo Level Prompt Generator) [9] describes an approach to single-line code auto completion task by providing additional contextual information such as information about other project files, in the prompt. We differ in that our task is *an entire method generation* i.e., unit test generation. Importantly, we provide a restricted set of additional contextual information due to the character limits on the prompt and input to the LLM. Monitor Guided Decoding [1] is a white box approach to LLMs that interferes at the output generation part of the LLM. In contrast, we do *not need* such an access to the internal weights of the LLM.

There has been a large body of work in automatically generating unit tests. These include non-Artificial Intelligence techniques such Evosuite [4] and Randoop [6]. Evosuite implements a search-based generation technique, while, Randoop implements a feedback-directed random technique.

Then we have unit test generation techniques based on machine and deep learning. Here unit test generation is treated in a manner similar to a natural language translation task (say English to French). The method under test (focal method) is provided as input, with its matching unit test as the output. Large number of such pairs are used in an encoder-decoder network to learn and generate unit tests for hitherto unseen focal methods [12].

Unit tests need an oracle that can specify the correct or the expected output from it. Automatically generating such oracles using neural networks has been explored in [3]. Once an oracle is known for a unit test, we need *assertions* that compare the expected and the actual output in different ways. Automatically generating such *assert* statements using transformers has been described in [13].

## 3 Background - Static Program Analysis

Static Program Analysis deals with analyzing the source code of a program without executing the program and observing the results (which is called dynamic program analysis). Static Program Analysis is heavily used in Software Engineering to detect problems such as null pointer exceptions; to automatically complete source code and so on.

Static Program Analysis involves several parts such as parsing the code, building Control Flow Graphs to explore control flow within the program, data flow analysis to detect data flow and so on.

Automated Java Parsers operate using the Backus Naur Form (BNF) Grammar of the Java Language and detect program entities such as method (function) declarations, method invocations (function calls). Parsers construct an Abstract Syntax Tree from the program source code text. Symbolic Solvers analyze the AST and informs about the types of the program entities (For example, in Listing 1, the getUnknownEmail method requires one parameter whose data type is Integer ).

## 4 Methodology: Program Analysis Guided Unit Test Generation via LLMs

Our approach along with the motivating example is shown in Listing 1. The method under test is shown on Line 1. Line 3 shows a simple prompt and Line 5 shows one line of interest in the LLM generated unit test in response to the prompt. Lines 7 and 8 depict *another* prompt augmented with information obtained by a static program analysis of the code (including the method under test and the project). Line 11 is the counterpart of line 5 and shows the new code generated.

Juxtaposing lines 5 and 11, we see that line 5 has two compilation errors, viz., the use of the *receiver object* i.e., *myClass* and the assumption about the parameter of *getUnknownEmail* being a *String*. In Line 11, due to the additional program analysis information, the LLM uses a correct object of the *Employee* class and correctly uses *anyInt()* to denote an *Integer* parameter.

We perform automated static program analysis using the Java Parser Library. For each focal method, we obtain the declaring class of the method, the complete signatures of the set of methods called from the focal method and the data type information about the fields used in the focal method. We then prompt the LLM to generate unit test(s) using the JUnit framework. We also prompt

```
1  String getEmail() { return isEmailUnknown() ? getUnknownEmail(number) : email; }
2
3  Simple Prompt: Please generate Unit Test
4    LLM Generated Unit Test (Only Line of Interest shown):
5      when(myClass.getUnknownEmail(anyString())).thenReturn("Unknown");
6
7  Program Analysis Enhanced Prompt: Please generate Unit Test. getUnknownEmail() defined in class
8  "Employee". Signature of getUnknownEmail = String getUnknownEmail(Integer)
9
10   LLM Generated Unit Test (Only Line of Interest shown):
11     when(employee.getUnknownEmail(anyInt())).thenReturn(" Unknown");
```

**Listing 1: Motivating Example: Line 1 is the Focal Method. Line 5 has two compilation errors which are remedied in Line 11 due to the program analysis information added to the prompt.**

the LLM to use *Mocking*, which is a way to isolate the dependencies of the focal method and concentrate on its behaviour.

It is especially important for mocking that the method signatures be known precisely in a strongly typed language like Java. Thus, this is where our addition of the program analysis information particularly helps.

## 5 Evaluation

**Experiment Subjects:** We evaluated our work on a large commercial Java Project from a telecom company. We also used the popular open source Java project, Guava.

**LLMs:** We used llama 7b/70b 2.0 and CodeLlama 34b based on llama 2.0 as the LLMs in our experiments as these have a commercial friendly license and have been shown to be amongst the better peforming LLMs. In addition, we have extended analysis on open-source code to GPT-4 for benchmarking. We are not allowed to use propietary code on GPT-4.

**Evaluation Metric:** Unit tests can be evaluated using a variety of factors such as syntactic and semantic correctness; assertions used, test smells and so on. However, before using these facets, test cases should actually be generated. We found that the baseline could not generate test cases for many focal methods. Thus, in this paper, we compare the number of focal methods for which one or more unit tests were generated by the baseline and our approach. We asked and answered the following research question:

**RQ1:** Does our approach of adding precise and concise program structural information to the prompt increase the number of focal methods for which one or more unit tests are generated?

### 5.1 Dataset and Experiments

We selected a random sub-project from our commercial Java project and this had 103 focal methods. We then performed the following two experiments: 1) A baseline approach as described in [10];and, (2) Our approach where for each focal method we provided only the focal method's signature and body along with the program structural information as shown in Listing 2 in Appendix.

For the open source project, Guava, we selected two random Java classes and repeated the above experiments with the 34 public methods in these classes.

### 5.2 Implementation Details

*5.2.1 LLM Parameters.* For inference using the LLMs we keep the input tokens limited to 1023 (the llama APIs mandate that it cannot be larger) and the full context length is default of 4096. Temperature is set to zero, topK is set to 50 and topP is set to 0.95. Other parameters are kept at their default. The models are running on a single A100 80 GB GPU and we use 8 bit quantization for inference.

*5.2.2 Experiment Setup.* We use the Java Parser library APIs to automatically parse the project Java code. For each method under test, we obtain the complete signatures of the invoked methods, such as, *int java.util.List getSize()*. We also obtain the declared type of any field accessed within the method. We augment the prompt with these additional program context information and provide the focal method to the LLM and collect the output i.e., the generated unit tests. We used llama and code llama as the LLMs as we needed an LLM that could be hosted locally and which had a commercial friendly license (Apache).

## 6 Results

### 6.1 Prompts and Sample Outputs

We present in this section a sample prompt for baseline and ours and output from the LLM on the prompts.

**Our Prompt:** Listing 2 shows a sample of the prompt used in our approach. It is for the method *ensureNonNegative* in the class *DoubleUtils* from Google's popular open source Java project, *Guava*. This is one of the two random classes we used from *Guava* in our open source evaluation.

```
[INST]
<<SYS>>
Generate Java unit test(s) for the given Java focal
     method. Mock method calls and fields using Mockito
     library. Use the JUnit framework and try to obtain
     high branch coverage.
<</SYS>>
FOCAL-METHOD-BEGIN
static double ensureNonNegative(double value) {
    checkArgument(!isNaN(value));
    return Math.max(value, 0.0);
}
FOCAL-METHOD-END
Declaring-Class-of-Method:
DoubleUtils
SIGNATURES-OF-METHOD-CALLS-WITHIN-FOCAL-METHOD:
```

| | llama7b | llama70b | codellama34b |
|---|---|---|---|
| Baseline | 37 | 9 | 7 |
| Ours | **102** | **88** | **63** |

**Table 1: 103 focal methods from commercial: For how many were unit tests generated?**

| | llama7b | llama70b | codellama34b | gpt-4 |
|---|---|---|---|---|
| Baseline | 20 | 20 | 1 | 32 |
| Ours | **30** | **31** | **19** | **34** |

**Table 2: 34 focal methods from open source: For how many were unit tests generated?**

| | Mean | Std. Dev. | Median |
|---|---|---|---|
| Baseline | 5295 | 3350 | 4056 |
| Ours | 559 | 516 | 464 |

**Table 3: Tokens in input**

| | Mean | Std. Dev. | Median |
|---|---|---|---|
| Baseline | 38 | 32 | 22 |
| Ours | 32 | 13 | 30 |

**Table 4: Time Taken in Seconds**

```
void com.google.common.base.Preconditions.checkArgument(
    boolean)
double java.lang.Math.max(double, double)
boolean java.lang.Double.isNaN(double)
FIELDS-USED-WITHIN-FOCAL-METHOD:
[/INST]
```

**Listing 2: Sample of our prompt: Method from Google Guava Project**

```
import static org.junit.Assert.*;
import org.junit.Test;
import org.mockito.Mockito;
public class DoubleUtilsTest {

    @Test
    public void testEnsureNonNegative() {
        // Test with positive input
        assertEquals(1.5, DoubleUtils.ensureNonNegative
     (1.5), 0.0001);

        // Test with negative input
        assertEquals(0.0, DoubleUtils.ensureNonNegative
     (-2.3), 0.0001);

        // Test with NaN input
        assertTrue(Double.isNaN(DoubleUtils.
     ensureNonNegative(Double.NaN)));
    }
}
```

**Listing 3: Output from the LLM for our prompt: One Unit Test Generated with correct assertions and suitably named identifiers and appropriate imports.**

**Baseline Prompt:** The baseline prompt for the same method, *ensureNonNegative* in the class *DoubleUtils* (guava project) is much longer than our prompt. It has the whole code prompted. [1]

**Output from baseline prompt:** The baseline prompt *did not* generate *any* unit tests. It simply regurgitated the input as the response.

## 6.2 Evaluation Results

Results are shown in Tables 1 and 2, for the commercial and open source projects respectively. It shows that across the different LLMs, our approach, is able to generate tests for more focal methods as compared with the baseline.

[1]The relevant code `DoubleUtils.java` is available at DoubleUtils.java

## 7 Discussion

Our hypothesis is that the baseline is not able to generate *any* test cases for *many* focal methods due to confounding factors such as increased context length, presenting extraneous information to the LLM. In contrast, our approach presents only *precise* and *concise* information to the LLM and thus can get the LLMs to be successful. To understand this phenomenon better, we obtained statistics on the input token lengths. The baseline has a mean and a median of 5295 and 4056 tokens, respectively. In contrast, the token length in our approach is noticeably smaller, with a mean and median 559 and 464 tokens, respectively. The statistics for tokens and time are presented in Table 3 and Table 4 respectively.

***Cost Factor:*** Many LLMs such as the OpenAI based ones have a token based charge system. In such cases, our approach would be further beneficial.

***Portability:*** Although we have demonstrated our approach on Java, it should be portable across languages, as we only need program analysis information which can be obtained automatically for every programming language out there using parsers and symbolic resolvers. The approach should also work for different complex methods as we have chosen methods randomly from open source and closed source (commercial projects).

## 8 Conclusion and Future Work

We described a novel approach to unit test generation using LLMs. The prompt of the LLM is guided by precise and concise information obtained automatically by program analysis. We evaluated our work on a commercial and an open source Java project. Our evaluation shows that our approach is able to generate unit tests for more focal methods as compared with the baseline approach, which presumably fails to due to the complexity and length of its input prompt and context. In future, we will deploy our solution in production and perform additional experiments using other metrics on commercial and open source software as well as extend to other programming languages. One limitation of this study is that we are only looking at generated test cases rather than their quality - this is a scope for future study. Also not all dependencies, especially those at runtime, can be analyzed via static program analysis - such limitations would affect our results too - however quantifying the impact of the same may be challenging. Further work involves extending our method to an In-context learning (ICL) or fine tuning of LLMs.

## References


[1] Lakshya Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K Lahiri, and Sriram Rajamani. 2023. Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context. In *Thirty-seventh Conference on Neural Information Processing Systems*.

[2] Ermira Daka and Gordon Fraser. 2014. A survey on unit testing practices and problems. In *2014 IEEE 25th International Symposium on Software Reliability Engineering*. IEEE, 201–211.

[3] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. 2022. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*. 2130–2141.

[4] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic Test Suite Generation for Object-Oriented Software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering* (Szeged, Hungary) *(ESEC/FSE '11)*. Association for Computing Machinery, New York, NY, USA, 416–419. https://doi.org/10.1145/2025113.2025179

[5] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]

[6] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: Feedback-Directed Random Testing for Java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion* (Montreal, Quebec, Canada) *(OOPSLA '07)*. Association for Computing Machinery, New York, NY, USA, 815–816. https://doi.org/10.1145/1297846.1297902

[7] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL]

[8] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. arXiv:2302.06527 [cs.SE]

[9] Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. 2023. Repository-level prompt generation for large language models of code. In *International Conference on Machine Learning*. PMLR, 31693–31715.

[10] Mohammed Latif Siddiq, Joanna C. S. Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2023. An Empirical Study of Using Large Language Models for Unit Test Generation. arXiv:2305.00418 [cs.SE]

[11] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL]

[12] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020).

[13] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. 2022. Generating accurate assert statements for unit test cases using pretrained transformers. In *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test*. 54–64.